

Path 2: Cyber Security 101 > Module 5: Networking > Room 3: Networking Core Protocols

<https://tryhackme.com/room/networkingcoreprotocols>

Networking Core Protocols:

Learn about the core TCP/IP protocols.

>> Task 1: Introduction

- ♦ Room Prerequisites:

To benefit from this room, we recommend that you know the following:

- ISO OSI model and layers
- TCP/IP model and layers
- Ethernet, IP, and TCP protocols

In other words, starting this room after **Networking Concepts** is the recommended approach.

- ♦ Learning Objectives:

By the time you finish this room, you will have learned about the following protocols:

- WHOIS
- DNS
- HTTP and FTP
- SMTP, POP3, and IMAP

>> Task 2: DNS: Remembering Addresses

Domain Name System (DNS), which is responsible for properly mapping a domain name to an IP address.

DNS operates at the Application Layer, i.e., Layer 7 of the ISO OSI model. DNS traffic uses UDP port 53 by default and TCP port 53 as a default fallback. There are many types of DNS records; however, in this task, we will focus on the following four:

- **A record:** The A (Address) record maps a hostname to one or more IPv4 addresses. For example, you can set **example.com** to resolve to **172.17.2.172**.
- **AAAA Record:** The AAAA record is similar to the A Record, but it is for IPv6. Remember that it is AAAA (quad-A), as AA and AAA would refer to a battery size; furthermore, AAA refers to Authentication, Authorization, and Accounting; neither falls under DNS.
- **CNAME Record:** The CNAME (Canonical Name) record maps a domain name to another domain name. For example, **www.example.com** can be mapped to **example.com** or even to **example.org**.
- **MX Record:** The MX (Mail Exchange) record specifies the mail server responsible for handling emails for a domain.

In other words, when you type **example.com** in your browser, your browser tries to resolve this domain name by querying the DNS server for the A record. However, when you try to send an email to **test@example.com**, the mail server would query the DNS server to find the MX record.

If you want to look up the IP address of a domain from the command line, you can use a tool such as **nslookup**.

Command Syntax: **nslookup www.example.com**

The query above led to four packets. In the terminal below, we can see that the first and third packets send DNS queries for the A and AAAA records, respectively. The second and fourth packets show the DNS query responses.

```
user@TryHackMe$ tshark -r dns-query.pcapng -Nn
```

```
1 0.000000000 192.168.66.89 → 192.168.66.1 DNS 86 Standard query 0x2e0f A
    www.example.com OPT
2 0.059049584 192.168.66.1 → 192.168.66.89 DNS 102 Standard query
    response 0x2e0f A www.example.com A
    93.184.215.14 OPT
3 0.059721705 192.168.66.89 → 192.168.66.1 DNS 86 Standard query 0x96e1
    AAAA www.example.com OPT
4 0.101568276 192.168.66.1 → 192.168.66.89 DNS 114 Standard query
    response 0x96e1 AAAA www.example.com
    AAAA 2606:2800:21f:cb07:6820:80da:af6b:8b2c
    OPT
```

Q1) Which DNS record type refers to IPv6?

Answer: AAAA

Q2) Which DNS record type refers to the email server?

Answer: MX

>> Task 3: WHOIS

You can register any available domain name for one or more years. You need to pay the annual fee, and you are required to provide accurate contact information (<https://www.icann.org/resources/pages/whois-data-accuracy-2017-06-20-en>) as the registrant. This information is part of the data available via WHOIS records and is available publicly. (Although written in uppercase, WHOIS is not an acronym; it is pronounced who is.) However, don't worry if you want to register a domain without revealing your contact information publicly; you can use one of the privacy services that hide all your information from the WHOIS records.

You can look up the WHOIS records of any registered domain name using one of the online services or via the command-line tool **whois**, available on Linux systems, among others. As expected, a WHOIS record provides information about the entity that registered a domain name, including name, phone number, email, and address.

Command: **whois example.com**

Q1) When was the x.com record created? Provide the answer in YYYY-MM-DD format.

Answer: 1993-04-02

Q2) When was the twitter.com record created? Provide the answer in YYYY-MM-DD format.

Answer: 2000-01-21

>> Task 4: HTTP(S): Accessing the Web

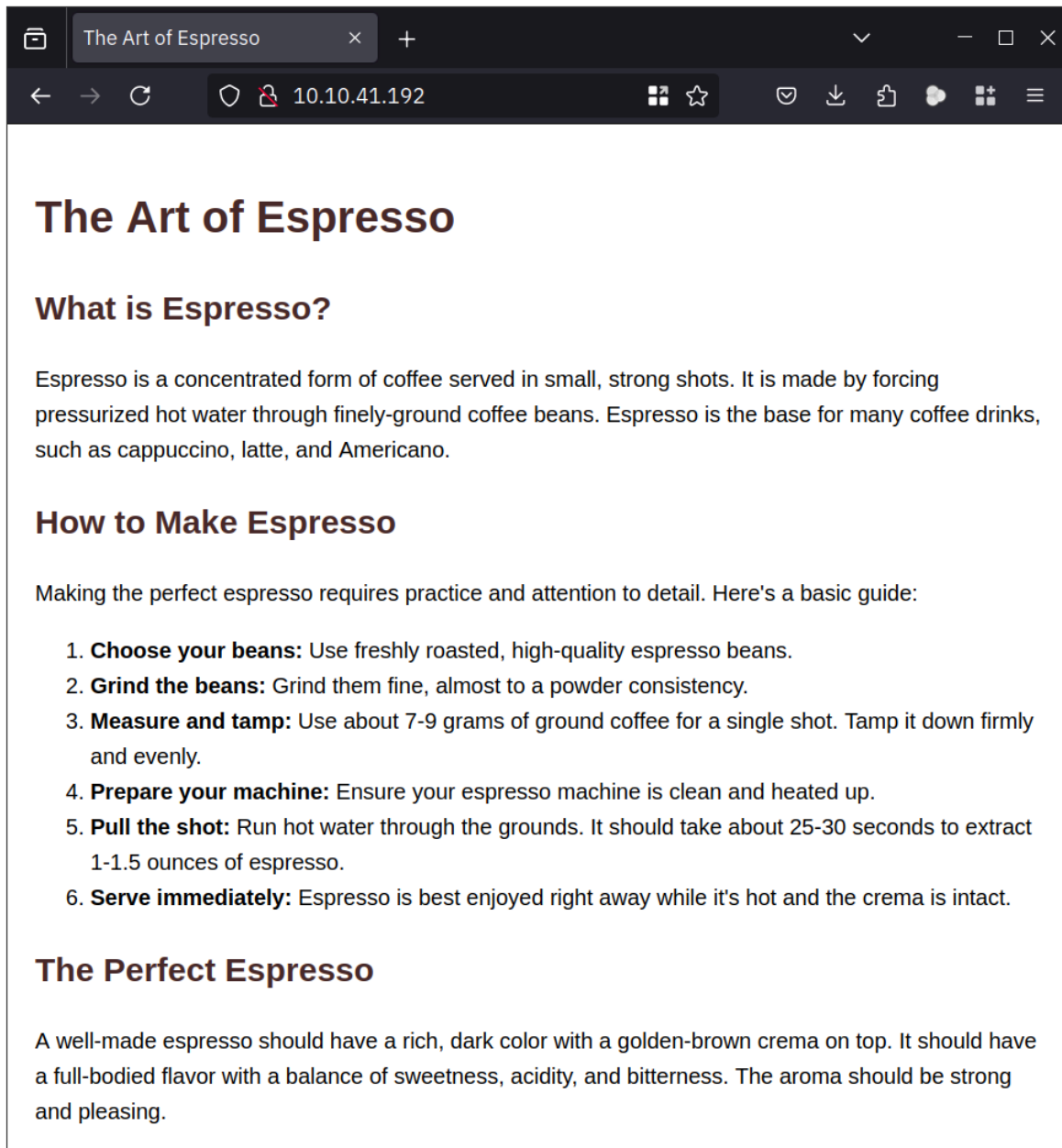
When you fire up your browser, you mainly use HTTP and HTTPS protocols. HTTP stands for Hypertext Transfer Protocol; the S in HTTPS stands for Secure. This protocol relies on TCP and defines how your web browser communicates with the web servers.

Some of the commands or methods that your web browser commonly issues to the web server are:

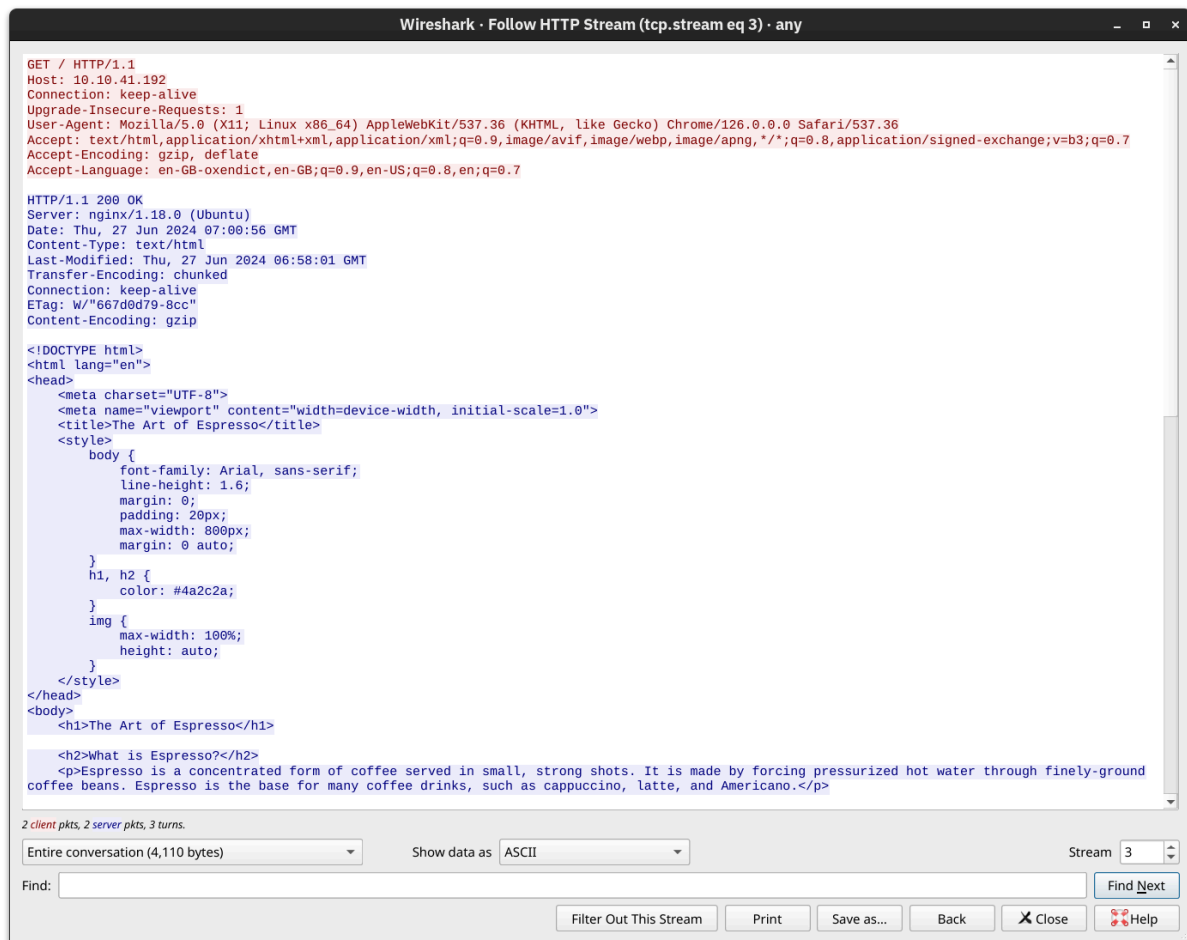
- **GET** retrieves data from a server, such as an HTML file or an image.
- **POST** allows us to submit new data to the server, such as submitting a form or uploading a file.
- **PUT** is used to create a new resource on the server and to update and overwrite existing information.
- **DELETE**, as the name suggests, is used to delete a specified file or resource on the server.

HTTP and HTTPS commonly use TCP ports **80** and **443**, respectively, and less commonly other ports such as **8080** and **8443**.

In the following example, we use our Firefox browser to access the web server on **10.80.132.179 (Machine_IP)**. Our browser fetches the web page and displays it perfectly; however, we are interested in what happens behind the scenes.



Using Wireshark, we can examine the exchange between the Firefox browser and the web server more closely. The screenshot below from Wireshark shows the text sent by our browser in **red** and the web server response in **blue**. As you can tell, a lot of information is exchanged between the client and the server that does not get rendered to the user. Examples include the web server version and when the page was last modified.



As you remember from **Networking Concepts**, we used the **telnet** client to connect to the web server running on **10.80.132.179** at port **80**. We had to send a couple of lines: **GET / HTTP/1.1** and **Host: anything** to get the page we wanted. (On some servers, you might get the file without sending **Host: anything**.) You can use this method to access any page and not just the default page **/**. To get **file.html**, you would send **GET /file.html HTTP/1.1**, for instance (**GET /file.html** might work depending on the web server in use). This approach is efficient for troubleshooting as you would be “talking HTTP” with the server.

Q1) Use **telnet** to access the file **flag.html** on **10.80.132.179** (**Machine_IP**). What is the hidden flag?

Answer: THM{TELNET-HTTP}

Hint: **telnet 10.8.132.179 80**
GET /file.html

>> Task 5: FTP: Transferring Files

Unlike HTTP, which is designed to retrieve web pages, File Transfer Protocol (FTP) is designed to transfer files. As a result, FTP is very efficient for file transfer, and when all conditions are equal, it can achieve higher speeds than HTTP.

Example commands defined by the FTP protocol are.

:

- **USER** is used to input the username
- **PASS** is used to enter the password
- **RETR** (retrieve) is used to download a file from the FTP server to the client.
- **STOR** (store) is used to upload a file from the client to the FTP server.

FTP server listens on TCP port 21 by default; data transfer is conducted via another connection from the client to the server.

In the terminal we executed the command `ftp 10.80.132.179` to connect to the remote FTP server using the local `ftp` client. Then we went through the following steps:

- Command `ftp 10.80.132.179`
- We used the username **anonymous** to log in
- We didn't need to provide any password
- Issuing `ls` returned a list of files available for download
- `type ascii` switched to ASCII mode as this is a text file
- `get coffee.txt` allowed us to retrieve the file we want

We used Wireshark to examine the exchanged messages more closely. The client's messages are in **red**, while the server's responses are in **blue**. Notice how various commands differ between the client and the server. For example, when you type `ls` on the client, the client sends **LIST** to the server. One last thing to note is that the directory listing and the file we downloaded are sent over a separate connection each.



Q1) Using the **FTP** client ftp on the AttackBox, access the FTP server at **10.80.132.179** and retrieve **flag.txt**. What is the flag found?
Answer: THM{FAST-FTP}

>> Task 6: SMTP: Sending Email

As with browsing the web and downloading files, sending email needs its own protocol. Simple Mail Transfer Protocol (SMTP) defines how a mail client talks with a mail server and how a mail server talks with another.

The analogy for the SMTP protocol is when you go to the local post office to send a package. You greet the employee, tell them where you want to send your package, and provide the sender's information before handing them the package. Depending on the country you are in, you might be asked to show your identity card. This process is not very different from an SMTP session.

Let's present some of the commands used by your mail client when it transfers an email to an SMTP server.

:

- **HELO** or **EHLO** initiates an SMTP session (Example: HELO client.thm)
- **MAIL FROM** specifies the sender's email address (Example: MAIL FROM: <user@client.thm>)
- **RCPT TO** specifies the recipient's email address (Example: RCPT TO: <strategos@server.thm>)
- **DATA** indicates that the client will begin sending the content of the email message
- . is sent on a line by itself to indicate the end of the email message

The terminal below shows an example of an email sent via **telnet**. The SMTP server listens on TCP port 25 by default.

Command: **telnet 10.82.164.159 25**

```
user@TryHackMe$ telnet 10.82.164.159 25
```

```
Trying 10.82.164.159...
```

```
Connected to 10.82.164.159.
```

```
Escape character is '^['.
```

```
220 example.thm ESMTP Exim 4.95 Ubuntu Thu, 27 Jun 2024 16:18:09 +0000
```

```
HELO client.thm
```

```
250 example.thm Hello client.thm [10.11.81.126]
```

```
MAIL FROM: <user@client.thm>
```

```
250 OK
```

```
RCPT TO: <strategos@server.thm>
```

```
250 Accepted
```

DATA

354 Enter message, ending with "." on a line by itself

From: user@client.thm

To: strategos@server.thm

Subject: Telnet email

Hello. I am using telnet to send you an email!

.

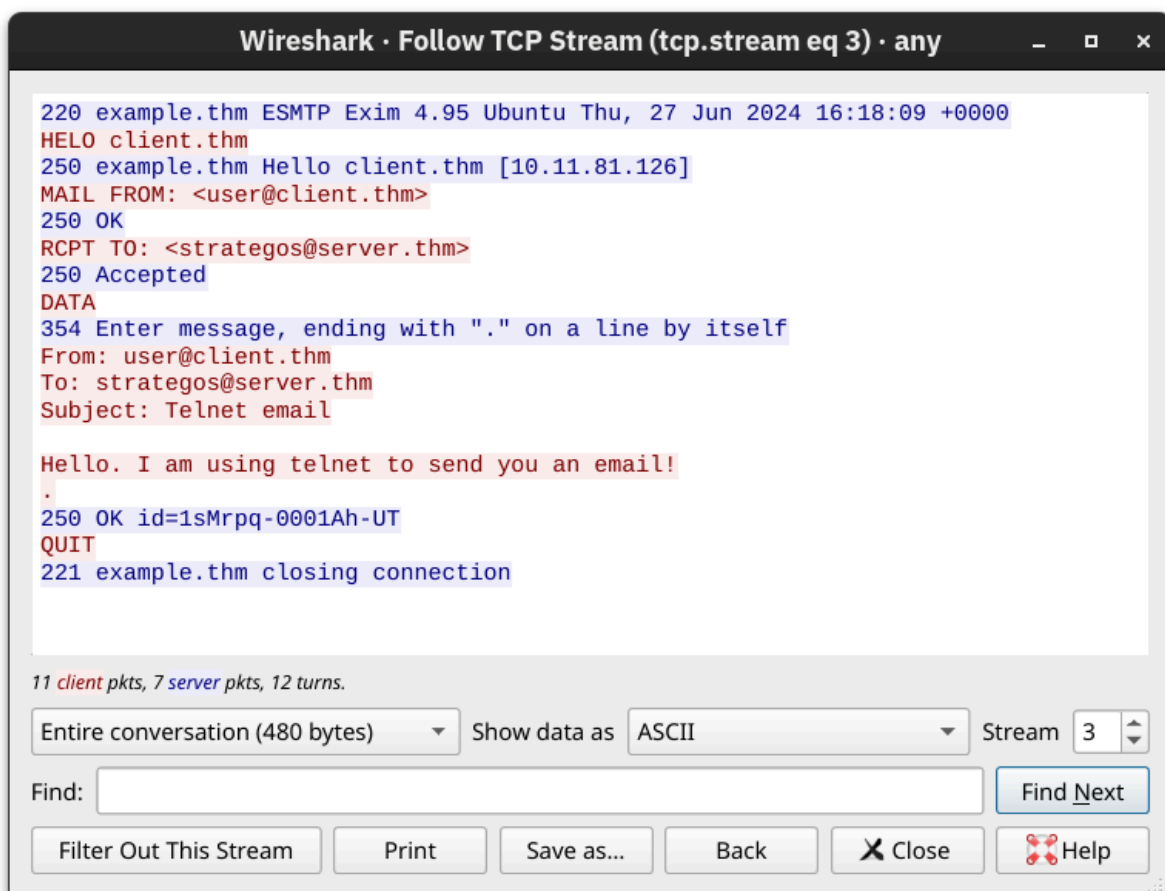
250 OK id=1sMrpq-0001Ah-UT

QUIT

221 example.thm closing connection

Connection closed by foreign host.

Obviously, sending an email using **telnet** is quite cumbersome; however, it helps you better understand the commands that your email client issues under the hood. The Wireshark capture shows the exchange in colours; the client's messages are in red, while the server's responses are in blue.



Q1) Which SMTP command indicates that the client will start the contents of the email message?

Answer: DATA

Q2) What does the email client send to indicate that the email message has been fully entered?

Answer: .

>> Task 7: POP3: Receiving Email

POP3: Post Office Protocol Version 3 (POP3) is an alternative protocol for receiving emails that downloads emails from the server to a local device. Using POP3, a recipient cannot access their emails again from a different device because they are stored locally and then deleted from the email server.

You've received an email and want to download it to your local mail client. The Post Office Protocol version 3 (POP3) is designed to allow the client to communicate with a mail server and retrieve email messages.

Without going into in-depth technical details, an email client sends its messages by relying on SMTP and retrieves them using POP3. SMTP is similar to handing your envelope or package to the post office, and POP3 is similar to checking your local mailbox for new letters or packages.

Some common POP3 commands are.

:

- **USER <username>** identifies the user
- **PASS <password>** provides the user's password
- **STAT** requests the number of messages and total size
- **LIST** lists all messages and their sizes
- **RETR <message_number>** retrieves the specified message
- **DELE <message_number>** marks a message for deletion
- **QUIT** ends the POP3 session applying changes, such as deletions

Command: **telnet 10.82.164.159 110**

We can see a POP3 session over telnet. Since the POP3 server listens on TCP port 110 by default, the command to connect to the TELNET port is telnet 10.82.164.159 110. The exchange below retrieves the email message sent in the previous task.

user@TryHackMe\$ **telnet 10.82.164.159 110**

Trying 10.82.164.159...

Connected to 10.82.164.159.

Escape character is '^['.

+OK [XCLIENT] Dovecot (Ubuntu) ready.

AUTH

+OK

PLAIN

.

USER strategos

+OK

PASS Pa\$\$123

+OK Logged in.

STAT

+OK 3 1264

LIST

+OK 3 messages:

1 407

2 412

3 445

.

RETR 3

+OK 445 octets

Return-path: <user@client.thm>

Envelope-to: strategos@server.thm

Delivery-date: Thu, 27 Jun 2024 16:19:35 +0000

Received: from [10.11.81.126] (helo=client.thm)

by example.thm with smtp (Exim 4.95)

(envelope-from <user@client.thm>)

id 1sMrpq-0001Ah-UT

for strategos@server.thm;

Thu, 27 Jun 2024 16:19:35 +0000

From: user@client.thm

To: strategos@server.thm

Subject: Telnet email

Hello. I am using telnet to send you an email!

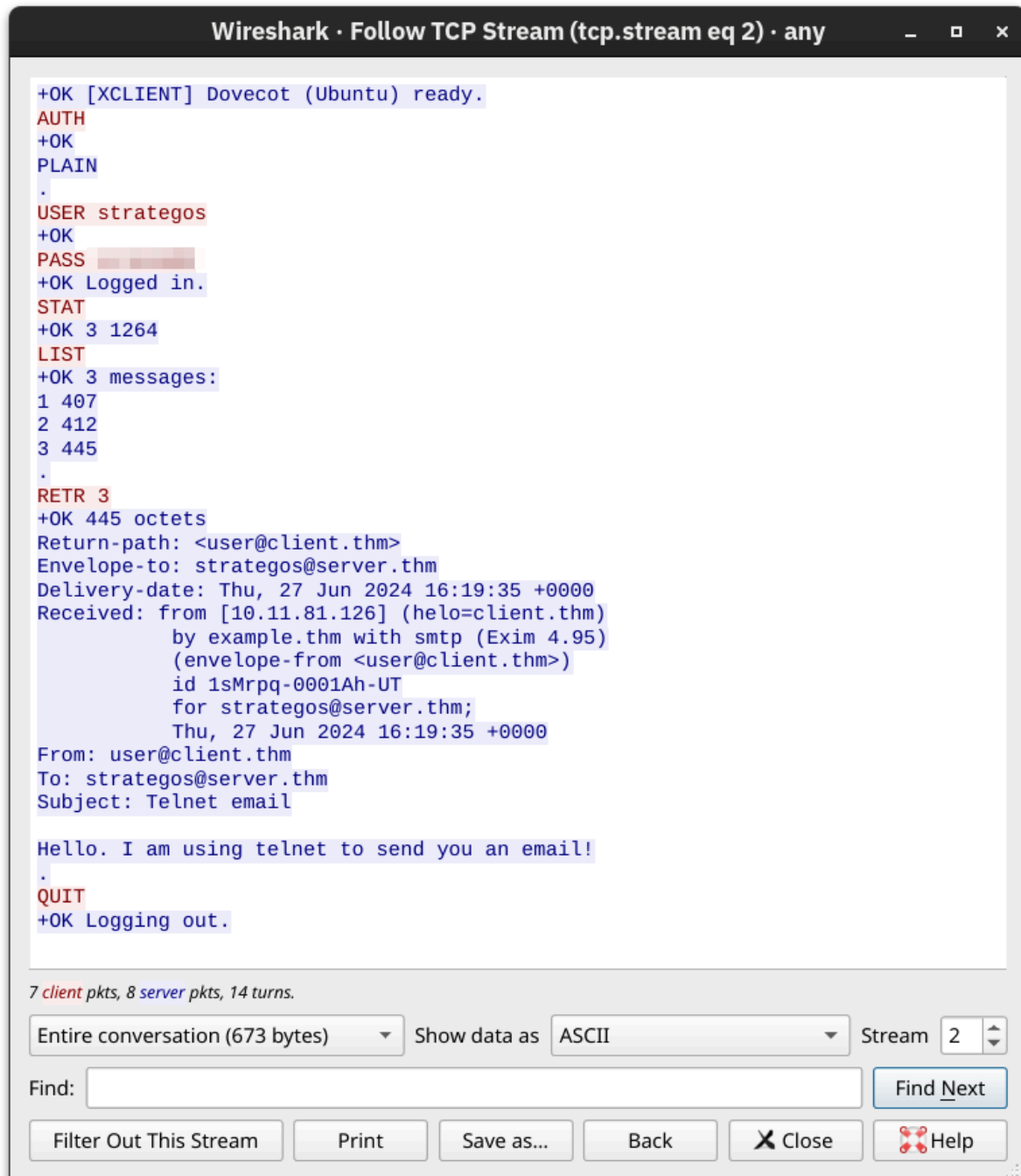
.

QUIT

+OK Logging out.

Connection closed by foreign host.

Someone capturing the network packets would be able to intercept the exchanged traffic. As per previous Wireshark captures, the commands in red are sent by the client, and the lines in blue are the server's. It is also clear that someone capturing the traffic can read the passwords.



Q1) Looking at the traffic exchange, what is the name of the POP3 server running on the remote server?

Answer: Dovecot

Q2) Use **telnet** to connect to **10.82.164.159**'s POP3 server. What is the flag contained in the fourth message?

Answer: THM{TELNET_RETR_EMAIL}

>> Task 8: IMAP: Synchronizing Email

IMAP: The Internet Message Access Protocol (IMAP) is a protocol for receiving email. Protocols standardize technical processes so computers and servers can connect with each other regardless of whether or not they use the same hardware or software.

POP3 is enough when working from one device, e.g., your favourite email client on your desktop computer. However, what if you want to check your email from your office desktop computer and from your laptop or smartphone? In this scenario, you need a protocol that allows synchronization of messages instead of deleting a message after retrieving it. One solution to maintaining a synchronized mailbox across multiple devices is Internet Message Access Protocol (IMAP).

IMAP allows synchronizing read, moved, and deleted messages. IMAP is quite convenient when you check your email via multiple clients. Unlike POP3, which tends to minimize server storage as email is downloaded and deleted from the remote server, IMAP tends to use more storage as email is kept on the server and synchronized across the email clients.

The IMAP protocol commands are more complicated than the POP3 protocol commands. We list a few examples below.

:

- **LOGIN** <username> <password> authenticates the user
- **SELECT** <mailbox> selects the mailbox folder to work with
- **FETCH** <mail_number> <data_item_name> Example **fetch 3 body[]** to fetch message number 3, header and body.
- **MOVE** <sequence_set> <mailbox> moves the specified messages to another mailbox
- **COPY** <sequence_set> <data_item_name> copies the specified messages to another mailbox
- **LOGOUT** logs out

Command: **telnet 10.82.164.159 143**

Knowing that the IMAP server listens on TCP port 143 by default, we will use **telnet** to connect to 10.82.164.159's port 143 and fetch the message we sent in an earlier task.

```
user@TryHackMe$ telnet 10.10.41.192 143
```

```
Trying 10.10.41.192...
```

```
Connected to 10.10.41.192.
```

```
Escape character is '^['.
```


* OK [CAPABILITY IMAP4rev1 SASL-IR LOGIN-REFERRALS ID ENABLE IDLE LITERAL+ STARTTLS AUTH=PLAIN] Dovecot (Ubuntu) ready.

A LOGIN strategos Pa\$\$123

A OK [CAPABILITY IMAP4rev1 SASL-IR LOGIN-REFERRALS ID ENABLE IDLE SORT SORT=DISPLAY THREAD=REFERENCES THREAD=REFS THREAD=ORDEREDSUBJECT MULTIAPPEND URL-PARTIAL CATENATE UNSELECT CHILDREN NAMESPACE UIDPLUS LIST-EXTENDED I18NLEVEL=1 CONDSTORE QRESYNC ESEARCH ESORT SEARCHRES WITHIN CONTEXT=SEARCH LIST-STATUS BINARY MOVE SNIPPET=FUZZY PREVIEW=FUZZY PREVIEW STATUS=SIZE SAVEDATE LITERAL+ NOTIFY SPECIAL-USE] Logged in

B SELECT inbox

* FLAGS (\Answered \Flagged \Deleted \Seen \Draft)

* OK [PERMANENTFLAGS (\Answered \Flagged \Deleted \Seen \Draft *)] Flags permitted.

* 4 EXISTS

* 0 RECENT

* OK [UNSEEN 2] First unseen.

* OK [UIDVALIDITY 1719824692] UIDs valid

* OK [UIDNEXT 5] Predicted next UID

B OK [READ-WRITE] Select completed (0.001 + 0.000 secs).

C FETCH 3 body[]

* 3 FETCH (BODY[] {445})

Return-path: <user@client.thm>

Envelope-to: strategos@server.thm

Delivery-date: Thu, 27 Jun 2024 16:19:35 +0000

Received: from [10.11.81.126] (helo=client.thm)

by example.thm with smtp (Exim 4.95)

(envelope-from <user@client.thm>)

id 1sMrpq-0001Ah-UT

for strategos@server.thm;

Thu, 27 Jun 2024 16:19:35 +0000

From: user@client.thm

To: strategos@server.thm

Subject: Telnet email

Hello. I am using telnet to send you an email!

)

C OK Fetch completed (0.001 + 0.000 secs).

D LOGOUT

* BYE Logging out

D OK Logout completed (0.001 + 0.000 secs).

Connection closed by foreign host.

The screenshot below shows the exchanged messages between the client and the server as seen from Wireshark. The client only needed to send four commands, shown in red, and the “long” server responses are shown in blue.

```
* OK [CAPABILITY IMAP4rev1 SASL-IR LOGIN-REFERRALS ID ENABLE IDLE LITERAL+ STARTTLS AUTH=PLAIN] Dovecot
(Ubuntu) ready.
A LOGIN strategos
A OK [CAPABILITY IMAP4rev1 SASL-IR LOGIN-REFERRALS ID ENABLE IDLE SORT SORT=DISPLAY THREAD=REFERENCES TH
READ=REFS THREAD=ORDEREDSUBJECT MULTIAPPEND URL-PARTIAL CATENATE UNSELECT CHILDREN NAMESPACE UIDPLUS LIS
T-EXTENDED I18NLEVEL=1 CONDSTORE QRESYNC ESEARCH ESORT SEARCHRES WITHIN CONTEXT=SEARCH LIST-STATUS BINAR
Y MOVE SNIPPET=FUZZY PREVIEW=FUZZY PREVIEW STATUS=SIZE SAVEDATE LITERAL+ NOTIFY SPECIAL-USE] Logged in
B SELECT inbox
* FLAGS (\Answered \Flagged \Deleted \Seen \Draft)
* OK [PERMANENTFLAGS (\Answered \Flagged \Deleted \Seen \Draft \*)] Flags permitted.
* 4 EXISTS
* 0 RECENT
* OK [UNSEEN 2] First unseen.
* OK [UIDVALIDITY 1719824692] UIDs valid
* OK [UIDNEXT 5] Predicted next UID
B OK [READ-WRITE] Select completed (0.001 + 0.000 secs).
C FETCH 3 body[]
* 3 FETCH (BODY[] {445}
Return-path: <user@client.thm>
Envelope-to: strategos@server.thm
Delivery-date: Thu, 27 Jun 2024 16:19:35 +0000
Received: from [10.11.81.126] (helo=client.thm)
        by example.thm with smtp (Exim 4.95)
        (envelope-from <user@client.thm>)
        id 1sMrpq-0001Ah-UT
        for strategos@server.thm;
        Thu, 27 Jun 2024 16:19:35 +0000
From: user@client.thm
To: strategos@server.thm
Subject: Telnet email

Hello. I am using telnet to send you an email!
)
C OK Fetch completed (0.001 + 0.000 secs).
D LOGOUT
* BYE Logging out
D OK Logout completed (0.001 + 0.000 secs).
```

4 client pkts, 5 server pkts, 8 turns.

Entire conversation (1,520 bytes) Show data as ASCII Stream 9

Find: Find Next

Filter Out This Stream Print Save as... Back Close Help

Q1) What IMAP command retrieves the fourth email message?
Answer: FETCH 4 body[]

>> Task 9: Conclusion

In the previous room, we discussed the TELNET protocol; this room focused on other fundamental protocols: DNS, HTTP, FTP, SMTP, POP3, and IMAP. With the protocols covered, we now have a better understanding of how domain names are resolved, how web pages are served, and how email is sent and received. Another primary purpose of this room is to give you a good understanding of how a protocol functions behind the graphical interfaces.

The table below summarizes the default port numbers of the protocols we have covered so far.

Protocol	Transport Protocol	Default Port Number
TELNET	TCP	23
DNS	UDP or TCP	53
HTTP	TCP	80
HTTPS	TCP	443
FTP	TCP	21
SMTP	TCP	25
POP3	TCP	110
IMAP	TCP	143